

Dynamic Question Flow & Answer Mapping

This documentation explains a **dynamic question-driven data collection system** built in Flutter. The system uses a question configuration list, enum-based question types, and centralized callbacks to map user input (text, speech, date, file, etc.) into a unified `UserAnswers` model.

 As requested, **full source code is NOT included**. Only **selected examples and explanations** are shown to clarify the architecture and logic.

Core Concepts Overview

The system is composed of four main parts:

1. **QuestionType enum** – defines what kind of input is required
 2. **QuestionModel** – represents a single question
 3. **Questions list** – drives the flow dynamically
 4. **Answer handlers** – map user input to `UserAnswers`
-

QuestionType Enum

`QuestionType` defines how each question should be rendered and handled in the UI.

Supported types: - `text` – normal text or speech input - `photo` – file/image upload - `date` – start & end date picker - `link` – URL input (e.g. LinkedIn) - `skip` – optional / skippable text - `creat` – custom creative input - `skill` – skills selector or tag input

This enum allows the UI to switch widgets dynamically based on question type.

QuestionModel (Model + Example)

`QuestionModel` is the **core model** that drives the entire dynamic question system. Every question shown to the user is an instance of this model.

Model Definition

This is the **actual model** used to describe a single question.

```
class QuestionModel {
    final int id;
    final String title;
    final QuestionType type;

    const QuestionModel({
        required this.id,
        required this.title,
        this.type = QuestionType.text,
    });
}
```

Model Responsibilities

`QuestionModel` answers three important questions:

1. Which data is being collected? → `id`
2. What does the user see? → `title`
3. How should input be taken? → `type`

This makes the flow **data-driven**, not hard-coded.

Example Usage

Below is a **realistic example** of how `QuestionModel` is used inside the question list.

```
final List<QuestionModel> questions = [
    QuestionModel(
        id: 1,
        title: "Please tell me your Name",
        type: QuestionType.text,
    ),
    QuestionModel(
        id: 7,
        title: "Start and End date",
        type: QuestionType.date,
    ),
    QuestionModel(
        id: 10,
        title: "Upload a professional photo",
        type: QuestionType.photo,
    ),
    QuestionModel(
```

```

    id: 13,
    title: "Please provide your LinkedIn profile URL",
    type: QuestionType.link,
  ),
];

```

Why `id` Matters More Than Index

Even though questions are shown sequentially, **answers are NOT mapped by index**.

Instead:

```
questions[currentQuestions].id
```

is used to decide **which field** in `UserAnswers` should be updated.

✓ This allows you to: - Reorder questions safely - Insert or remove questions - Keep backend contracts stable

----- ----- -----		<code>id</code>		<code>int</code>		Unique identifier used for answer mapping		<code>title</code>		<code>String</code>	
Question text shown to the user						<code>type</code>		<code>QuestionType</code>		Determines input widget & behavior	

Why `id` is Important

The `id` is **not just an index**. It is used to: - Map speech/text input to `UserAnswers` - Keep data stable even if questions are reordered - Avoid tight coupling between UI order and data logic

Question List (Flow Controller)

A single ordered `List<QuestionModel>` defines the entire questionnaire flow.

Why this approach works well

- Easy to **add, remove, or reorder** questions
- UI becomes fully **dynamic**
- Same rendering logic can handle many question types

Example question intent: - Personal info (name, email, phone) - Job & company details - Date-based employment period - Optional sections (responsibilities, links) - Media input (photo upload) - Skills, education, references

Current Question Index

```
int currentQuestions = 0;
```

- Tracks which question is currently active
- Used to fetch:
 - Current question title
 - Question type
 - Question ID for answer mapping

Speech-to-Text Handling

Speech input is handled centrally and routed to the correct answer field based on the **current question ID**.

Conceptual Flow

1. User speaks
2. Speech result is returned as text
3. Text is mapped to the correct field in `UserAnswers`

Mapping Strategy

Instead of binding UI directly to fields, a `switch` on `question.id` is used.

Example mapping logic (simplified):

```
switch (questions[currentQuestions].id) {  
  case 1:  
    userAnswers.name = text;  
    break;  
  case 2:  
    userAnswers.email = text;  
    break;  
  case 3:  
    userAnswers.mobileNumber = text;  
    break;  
}
```

✓ This keeps UI generic and logic centralized

Listening State Handling

The listening state is tracked separately:

- Used to update UI (mic animation, glow, etc.)
- Reset when moving to the next question

```
bool _isListening = false;
```

Date Input Handling

Date-based questions (like employment period) use a **dual-field handler**:

- `startDate`
- `endDate`

The handler safely updates only provided values.

Design benefit: - Works with partial updates - Supports "currently working" logic

Checkbox Logic (Currently Working)

Checkbox input is handled independently and stored in `UserAnswers`.

Typical use case: - Disable end date when checked - Reflect current employment status

File / Photo Input Handling

File picker results are mapped directly:

- Stores only the file path
- Upload can be handled later (backend step)

This keeps UI fast and memory-efficient.



Navigating Between Questions

The flow is controlled using a single method:

Responsibilities

- Increment question index
- Reset speech & listening state
- Reset speech widget key
- Debug-print collected answers

When the last question is reached: - All answers are available in `UserAnswers` - Data is ready for API submission

Data Collection Result

All user input is stored in a single model:

```
UserAnswers
├─ name
├─ email
├─ mobileNumber
├─ jobTitle
├─ dates
├─ photoPath
├─ skills
├─ education
└─ references
```

This makes backend integration simple: - Convert to JSON - Send once after completion

Architecture Benefits

- ✓ Fully dynamic question flow
 - ✓ Scalable (add new question types easily)
 - ✓ Clean separation of UI and data
 - ✓ Speech, text, date, and file inputs unified
 - ✓ Ideal for onboarding, CV builder, or form wizards
-



Possible Enhancements

- Replace `switch` with a **map-based strategy**
- Add validation per `QuestionType`
- Persist progress locally
- Animate question transitions

If you want, next I can: - Design the **dynamic UI renderer** per `QuestionType` - Refactor this into a **clean architecture** pattern - Create a **flow diagram** for this system

Just tell me ♀